Projeto e Análise de Algoritmos

SOMAS +

Computando o custo de trechos de algoritmos +

Divisão e Conquista

Universidade Federal do Amazonas
Departamento de Ciência da Computação

Projeto e Análise de Algoritmos

SOMATÓRIOS

Universidade Federal do Amazonas
Instituto de Computação



REVISÃO

Notação Sigma

$$\underbrace{1+1+1+\ldots+1}_{n} = \sum_{i=1}^{n} 1$$

$$\underbrace{1+2+3+\ldots+n}_{n} = \sum_{i=1}^{n} i$$

$$\underbrace{1+2+4+8+\ldots+2^n}_{n} = \underbrace{2^0+2^1+2^2+2^3+\ldots+2^n}_{n+1} =$$

$$\sum_{i=0}^{n} 2^i$$



COMO LIDAR COM SOMATÓRIOS ?

Resolvendo somatórios
Algumas técnicas úteis

- Procurar resposta e conferir
  - Vale principalmente para o dia-a-dia.
- Indução
- Perturbação
- Somas Telescópicas
- Usar integrais

Prova por indução

- Deseja-se mostrar que uma propriedade P é verdadeira para todo inteiro $n \geq n_0$
- **Base**:
  - provar que P é verdadeiro para n_0
- **Passo indutivo**:
  - provar que
    - se P é verdadeiro para todo k
    - tal que $n_0 \leq k \leq n-1$
    - então P é verdadeiro para n

Prova por indução (2)

- Exemplo

$$S(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \text{ for } n \geq 1$$

- Base

$$S(1) = \sum_{i=1}^{1} i = \frac{1(1+1)}{2}$$

Prova por Indução (3)

- Passo indutivo

$$S(k) = \sum_{i=1}^{k} i = \frac{k(k+1)}{2} \quad 1 \leq k \leq n-1$$

$$S(n) = \sum_{i=1}^{n} i = \sum_{i=1}^{n-1} i + n = S(n-1) + n =$$

$$= (n-1)\frac{((n-1)+1)}{2} + n = \frac{(n^2 - n + 2n)}{2} =$$

$$= \frac{n(n+1)}{2}$$

Somatórios (2)

- Progressão aritmética

$$\sum_{i=0}^{n} i = 1+2+3+\ldots+n = \frac{n^2+n}{2}$$

- Progresão Geométrica
  - Seja um inteiro n_0 e um real $0 < a \neq 1$

$$\sum_{i=0}^{n} a^i = 1+a+a^2+\ldots+a^n = \frac{1-a^{n+1}}{1-a}$$

  - Progressões geométricas apresentam crescimento exponencial

Algumas propriedades

$$\sum_{k \in K} c a_k = c \sum_{k \in K} a_k$$

Distributiva

$$\sum_{k \in K} (a_k + b_k) = \sum_{k \in K} a_k + \sum_{k \in K} b_k$$

Associativa

$$\sum_{k \in K} a_k = \sum_{p(k) \in K} a_{P(k)}$$

Comutativa
(P é uma permutação)

Perturbação

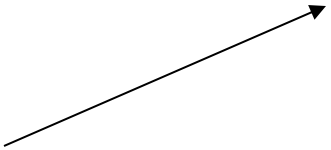
- Escrever Sn+1 tirando o primeiro termo e igualar a Sn+1 tirando o último termo
- Encontrar Sn dos dois lados da equação
- Isolar Sn para encontrar a fórmula fechada para o somatório

Perturbação

$$S_n = \sum_{k=0}^{n} a_k$$

$$S_{n+1} = S_n + a_{n+1}$$

$$S_{n+1} = a_0 + \sum_{k=1}^{n+1} a_k = a_0 + \sum_{k=0}^{n} a_{k+1}$$



O truque agora consiste em quebrar o termo do somatório para chegar a Sn

Perturbação

$$S_n = \sum_{k=0}^{n} c^k$$

$$S_{n+1} = S_n + c^{n+1}$$

$$S_{n+1} = c^0 + \sum_{k=1}^{n+1} c^k = 1 + \sum_{k=0}^{n} c^{k+1} = 1 + c\sum_{k=0}^{n} c^k = 1 + cS_n$$

$$1 + cS_n = S_n + c^{n+1}$$

$$S_n = \frac{c^{n+1} - 1}{c - 1}$$

Perturbação

- Dá pra resolver somatórios importantes, como (tente sempre resolver i^{k+1} para achar i^k, ex. i^2, acha i^1):

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$



Soma telescópica

$$\sum_{i=k}^{n} a_i - a_{i-1} = a_n - a_{k-1}$$

$$\sum_{i=2}^{n} (i^2 - (i-1)^2) = n^2 - (1)^2$$

$$\sum_{i=2}^{n} i^2 - \sum_{i=2}^{n} (i-1)^2$$

$$+2^2 + 3^2 + \ldots + (n-2)^2 + (n-1)^2 + n^2$$

$$-1^2 \quad -2^2 - 3^2 - \ldots - (n-2)^2 - (n-1)^2$$

Demais parcelas internas são somadas e subtraídas, anulando-se

Soma telescópica

$$\sum_{i=k}^{n} a_i - a_{i-1} = a_n - a_{k-1}$$

Demais parcelas internas são somadas e subtraídas, anulando-se

Soma telescópica

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \sum_{i=1}^{n} \left(\frac{1}{i} - \frac{1}{i+1}\right) = -\sum_{i=1}^{n} \left(\frac{1}{i+1} - \frac{1}{i}\right)$$

Fazendo 1/(i+1) ser a_i, 1/(i) seria a_{i-1}, resultado seria a_n=1/(n+1) menos a_0=1/1…

$$-\sum_{i=1}^{n} \left(\frac{1}{i+1} - \frac{1}{i}\right) == -\left(\frac{1}{n+1} - 1\right) = 1 - \frac{1}{n+1} = \frac{(n+1)-1}{n+1} = \frac{n}{n+1}$$

Integrais

- Integrais podem ser usadas para encontrarmos o valor aproximado de um somatório, indicando seu comportamento assintótico.
- Exemplo:

$$\sum_{i=0}^{n} i = O(\int_{0}^{n} i) = O(n^2)$$

Somatórios Importantes

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

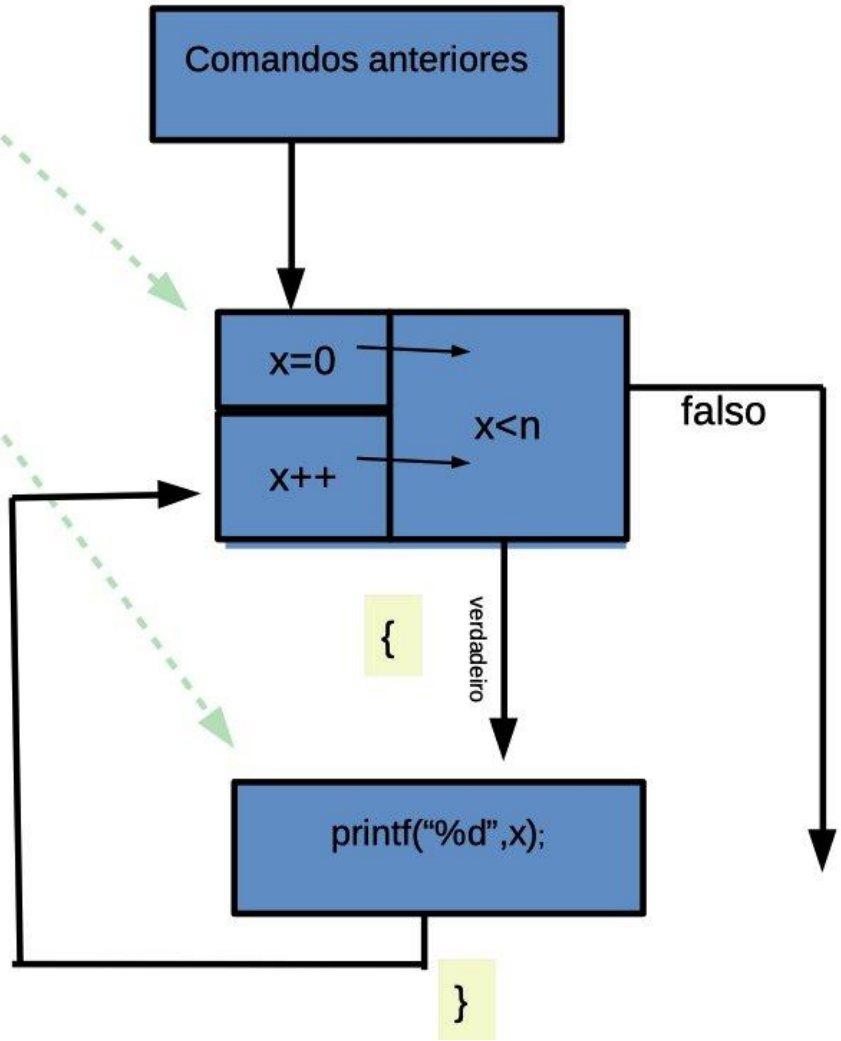
$$\sum_{i=1}^{n} \frac{1}{i} = \ln(n) + O(1)$$

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1}$$

$$\sum_{i=a}^{n} 1 = (n - a) + 1$$

Analisando custos...

```
for(x=0; x< n; x++) {
    printf("%d",x);
}
```



Cada vez que passa no printf
Custo 1
Custo total:
1+1+...+1

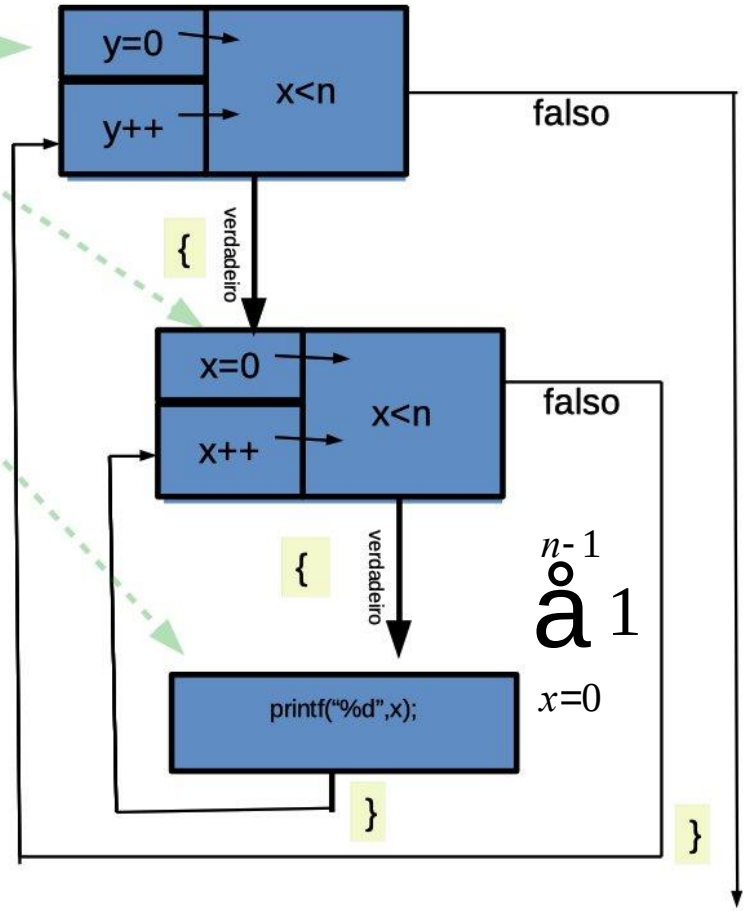
Quantos 1s tem nessa soma ?

$$\underbrace{1+1+1+1\ldots+1}_{n}$$

$$\sum_{x=0}^{n-1} 1 = (n-1) - 0 + 1 = n$$

Loops aninhados...

```
for(y =0; y < n; y++) {
    for(x=0; x< n; x++) {
       printf("%d",x);
    }
}
```



Cada vez que passa no printf
Custo 1
Custo total da parte de dentro:
1+1+...+1, n vezes

No loop de fora:
n+n+n+...+n, n vezes

$$\sum_{y=0}^{n-1} \sum_{x=0}^{n-1} 1$$

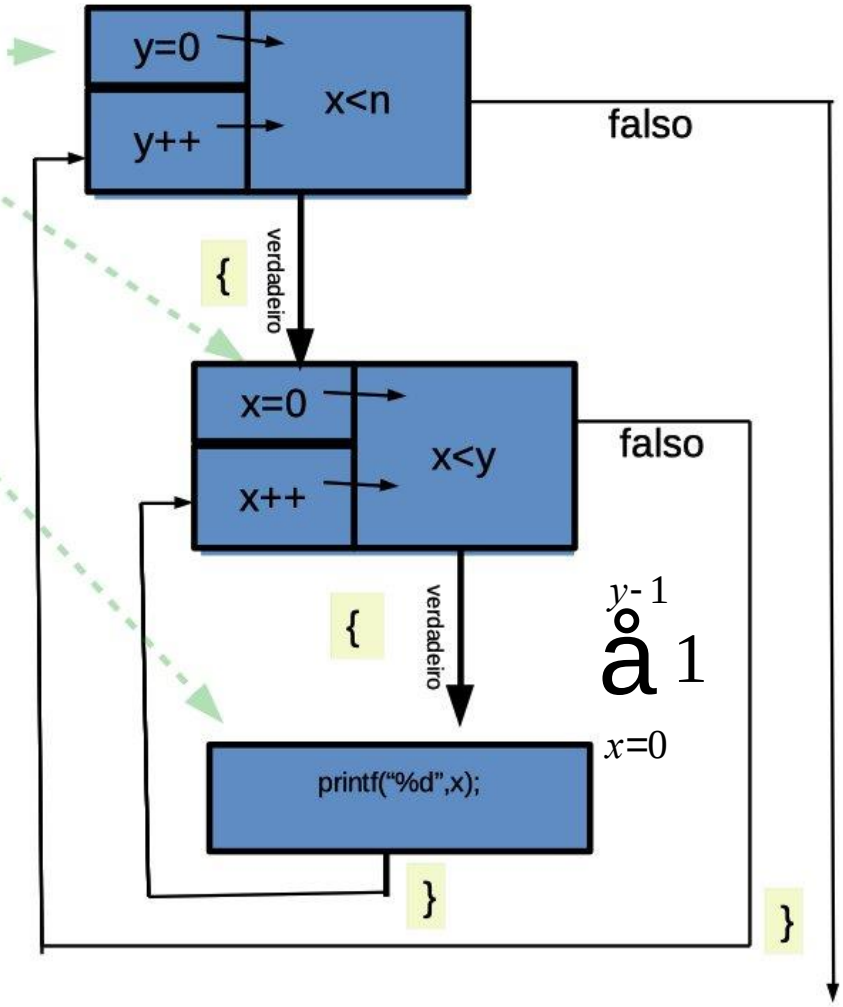
$$\sum_{x=0}^{n-1} 1$$

$x=0$

Calculando o custo

$$\sum_{y=0}^{n-1}\sum_{x=0}^{n-1} 1 = \sum_{y=0}^{n-1} n = n\sum_{y=0}^{n-1} 1 = n.n = n^2$$

```
for(y =0; y < n; y++) {
    for(x=0; x< y; x++) {
       printf("%d",x);
    }
}
```



$$\sum_{y=0}^{n-1}\sum_{x=0}^{y-1} 1$$

Calculando o custo

$$\sum_{y=0}^{n-1}\sum_{x=0}^{y-1} 1 = \sum_{y=0}^{n-1} y = \frac{(n-1).n}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$\sum_{x=0}^{y-1} 1 = y$$

Somatórios

- O tempo de execução da ordenação por inserção é determinado pelos loops aninhados:

```
for j←2 to length(A)
    key←A[j]
    i←j-1
    while i>0 and A[i]>key
        A[i+1]←A[i]
        i←i-1
    A[i+1]←key
```

- Loops aninhados correspondem a somatórios aninhados

Exemplo

Ordenação por Inserção

```
for j←2 to |A|
    key←A[j]
    i←j-1
    while i>0 and A[i]>key
        A[i+1]←A[i]
        i←i-1
    A[i+1]←key
```

$|A| = 2 \rightarrow 1$
$|A| = 3 \rightarrow 1 + 2$
$|A| = 4 \rightarrow 1 + 2 + 3$
...
$|A| = n \rightarrow 1 + 2 + 3 + \ldots + n\text{-}1$

$$\sum_{j=2}^{n} (j - 1)$$

Determine a função de custo

```
FOR(x=0; x< n; x++) {
    FOR(y=0; y<x; y++) {
        FOR(z=y; z<x; z++) {
            faz_algo;
        }
    }
}
```

Determine a função de custo

```
FOR(x=1; x< n; x=x*2) {
    FOR(y=0; y<x; y++) {
        faz_algo;
    }
}
```

Determine a função de custo

```
FOR(x=0; x<= n; x=x+2) {
    FOR(y=0; y<x; y++) {
        faz_algo;
    }
}
```



Falando um pouco de projeto de algoritmos...

Divisão e Conquista

- Método usado para projeto de algoritmos:
  - **Divisão**: Se o tamanho da entrada é muito grande para aplicação de uma solução simples, dividir o problema e dois ou mais subproblemas disjuntos
  - **Conquista**: Usar o método recursivamente para resolver os subproblemas
  - **Combinação**: Tomar as soluções dos subproblemas e combina-las para compor uma solução para o problema original.

MergeSort

- **Divisão**: Se S tem $n \geq 2$ elementos, dividir seus elementos em duas subsequencias S_1 e S_2 com respectivamente $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ elementos.
- **Conquista**: Ordernar as subsequencias S_1 e S_2 usando MergeSort.
- **Combinação**: Intercale os elementos de S_1 e S_2 de forma a obter uma seqüência ordenada



Como Analisar Algoritmos Recursivos ?

Recorrências

- As **chamadas recursivas** nos algoritmos podem ser descritas usando-se equações(ou inequações) de **recorrência**

- São funções descritas em termos dos seus valores para entradas menores.

Recorrências (2)

- Exemplo: Busca Binária

```
BuscaBin(A[1…n],q)
    if n=1
       then if A[n]=q then return n
                      else return 0

     k←(n+1)/2
     if q < A[k] then BuscaBin(A[1…k-1],q)
                 else BuscaBin(A[k…n],q)
```

$$T(n) = \begin{cases} 1 & \text{if } n = 1; \\ 1 + T(n/2) & \text{if } n > 1; \end{cases}$$

Recorrências (3)

- Exemplo: MergeSort

```
Merge-Sort(A, p, r)
    if p < r then
        q←(p+r)/2
        Merge-Sort(A, p, q)
        Merge-Sort(A, q+1, r)
        Merge(A, p, q, r)
```

$$t(n) = \begin{cases} 1 & se \quad n \leq 1 \\ t(n/2) + t(n/2) + t_{merge}(n) & se \quad n > 1 \end{cases}$$

Recorrências (3)

- Exemplo: MergeSort

```
Merge-Sort(A, p, r)
    if p < r then
        q←(p+r)/2
        Merge-Sort(A, p, q)
        Merge-Sort(A, q+1, r)
        Merge(A, p, q, r)
```

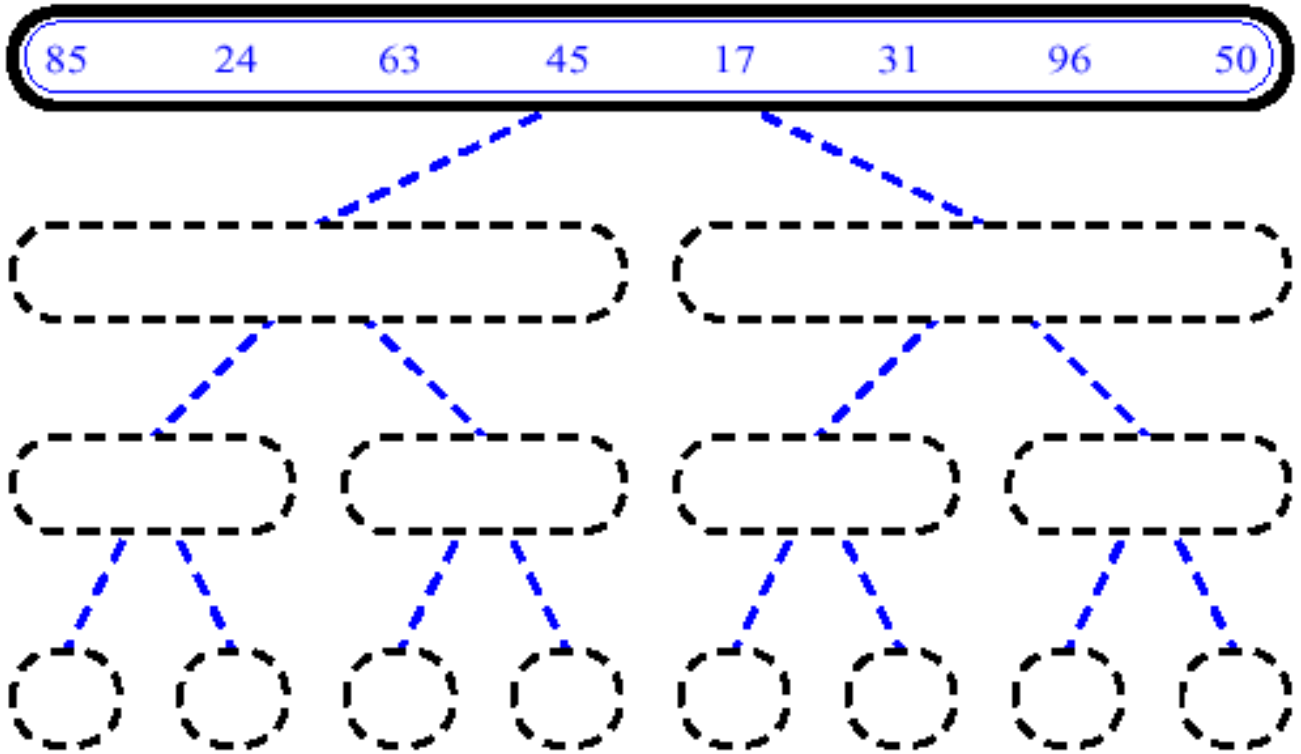
$$t(n) = \begin{cases} 1 & se \quad n \leq 1 \\ 2t(n/2) + n & se \quad n > 1 \end{cases}$$

Merge Sort: Algoritmo

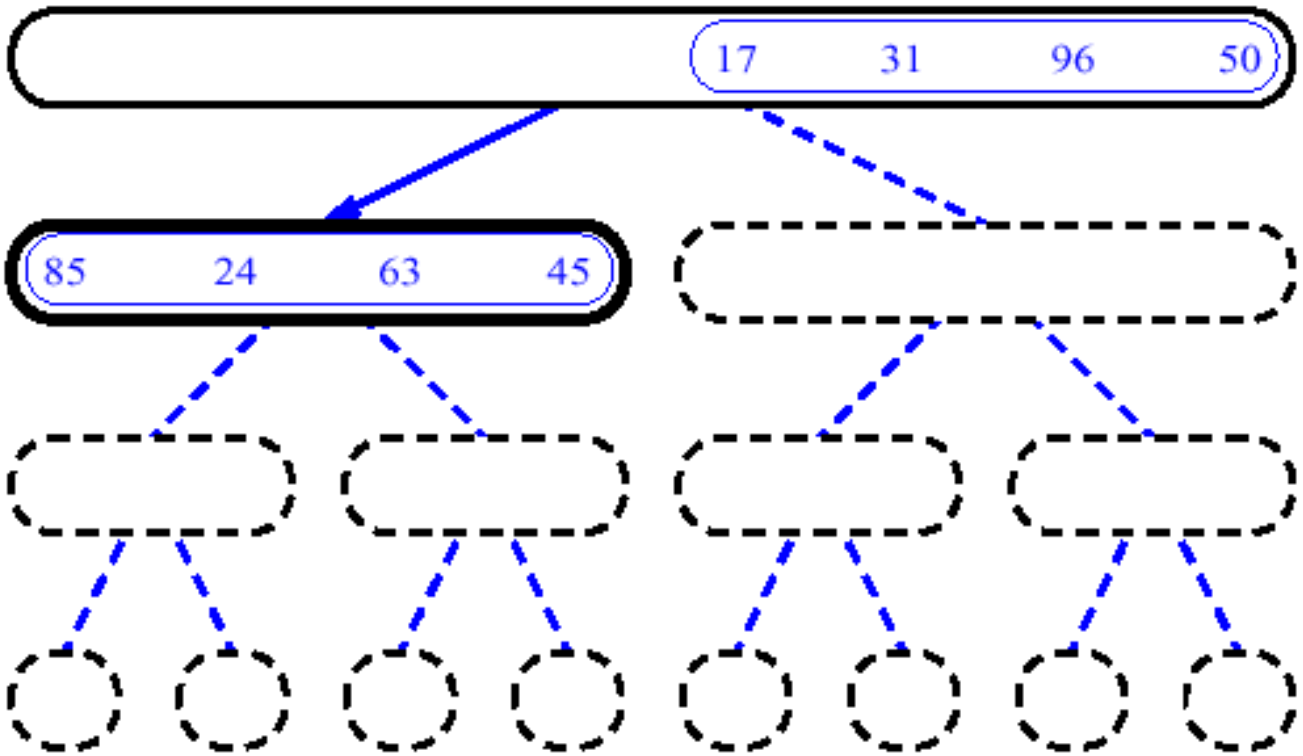
```
Merge-Sort(A, p, r)
    if p < r then
        q←(p+r)/2
        Merge-Sort(A, p, q)
        Merge-Sort(A, q+1, r)
        Merge(A, p, q, r)
```

```
Merge(A, p, q, r)
   Retire o menor entre o menor dos elementos de
A[p…q] e A[q+1…r] e acrescente ao resultado. Repita
até que as duas sub-sequências estejam vazias.
```

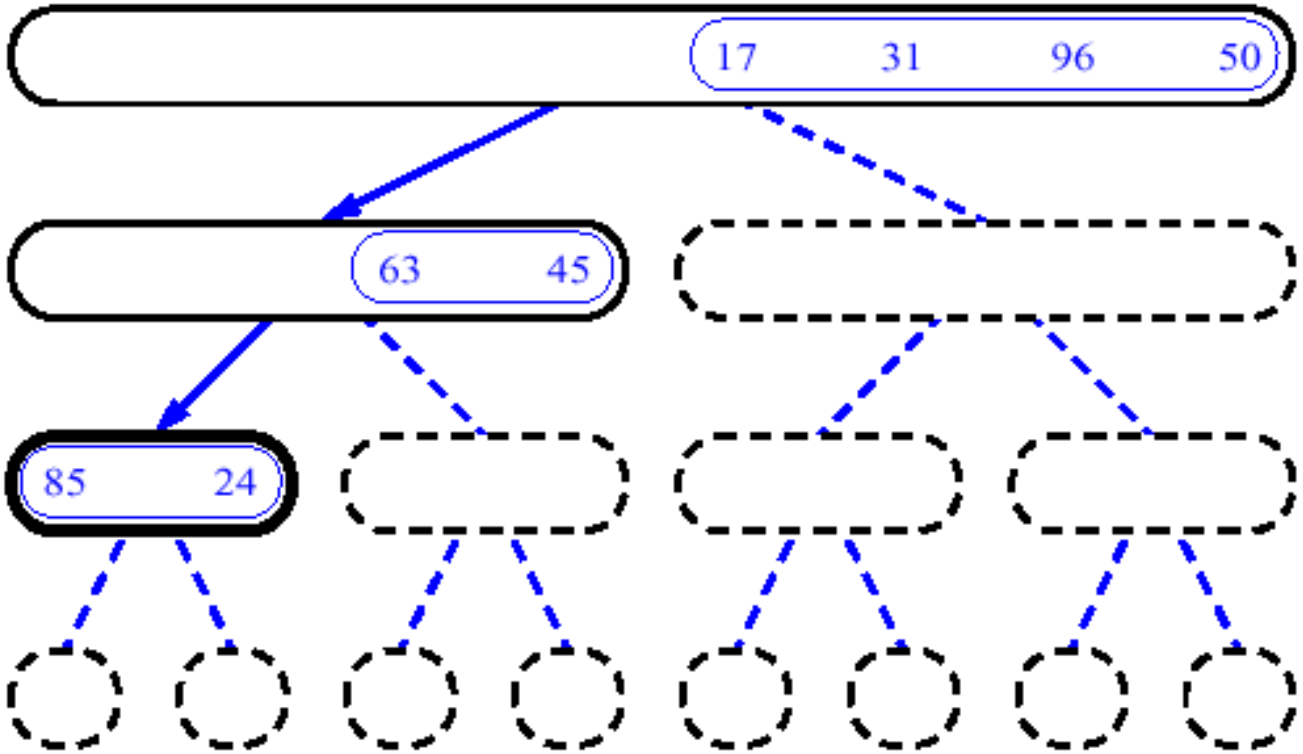
MergeSort (Exemplo) - 1



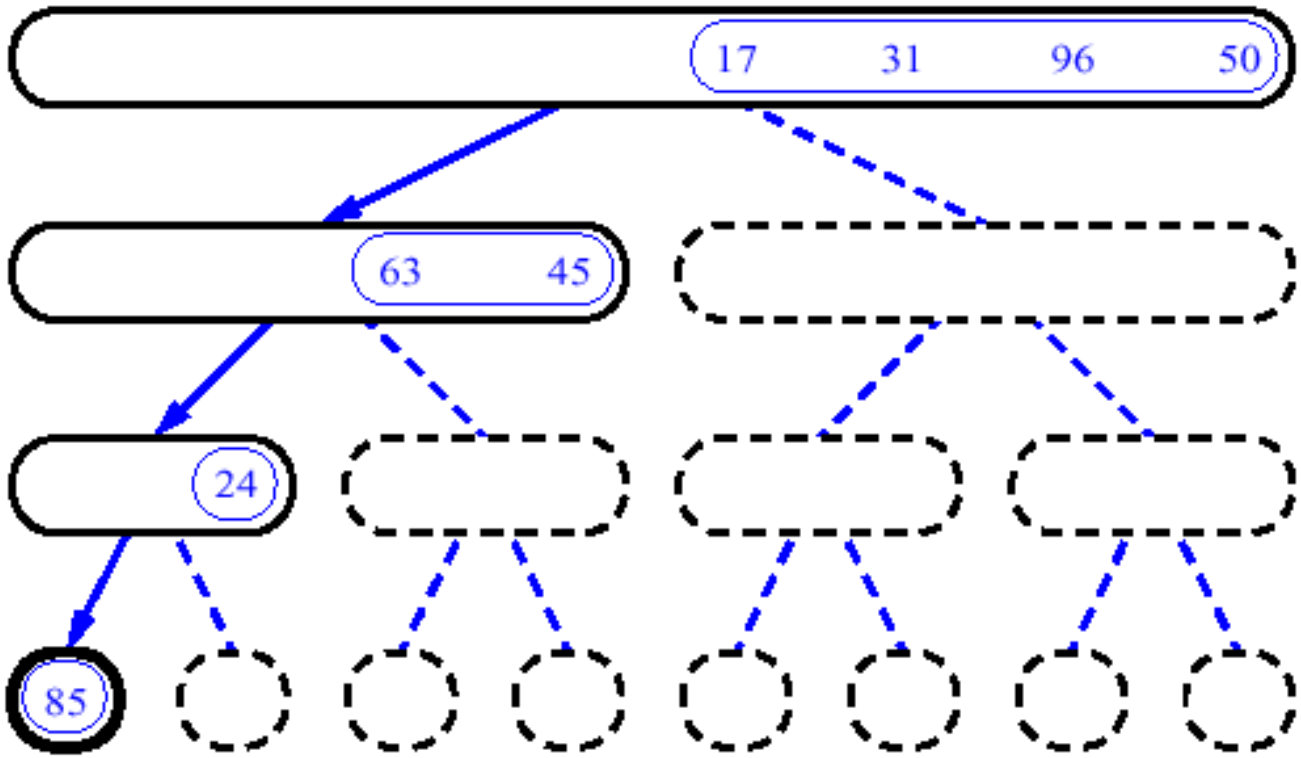
MergeSort (Exemplo) - 2



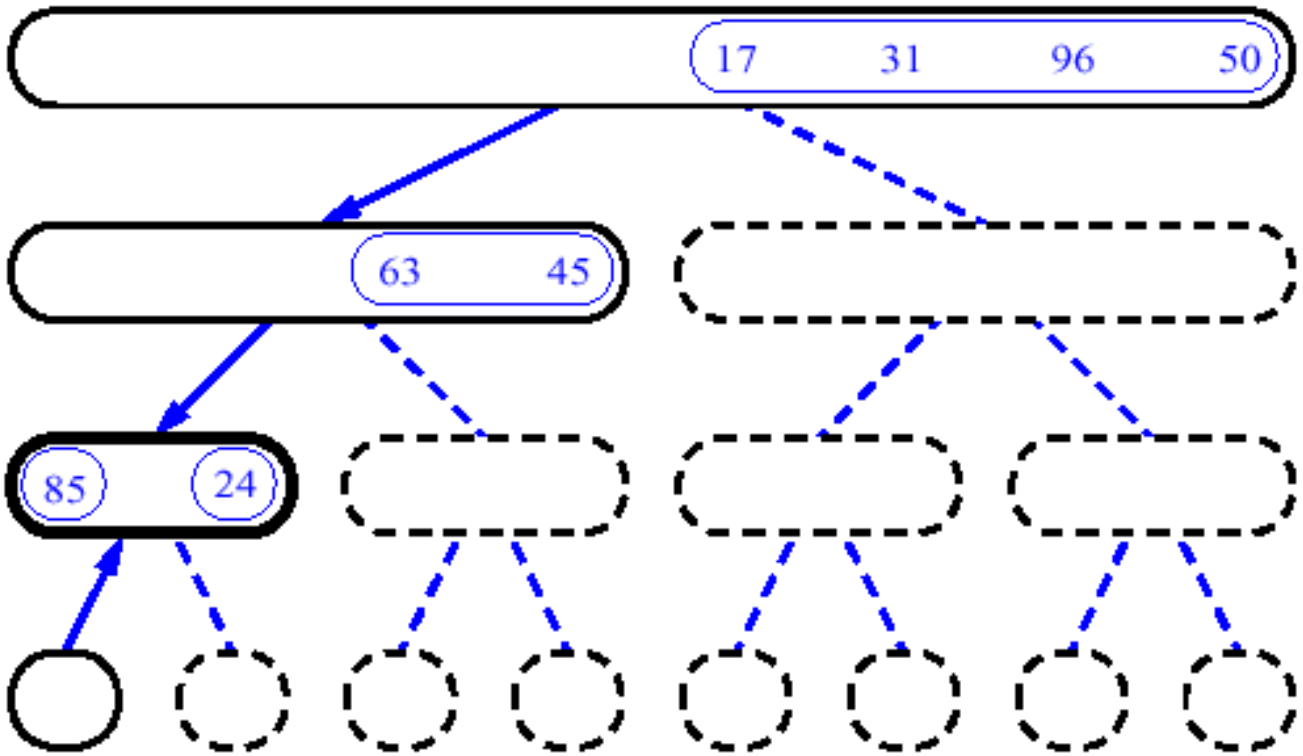
MergeSort (Exemplo) - 3



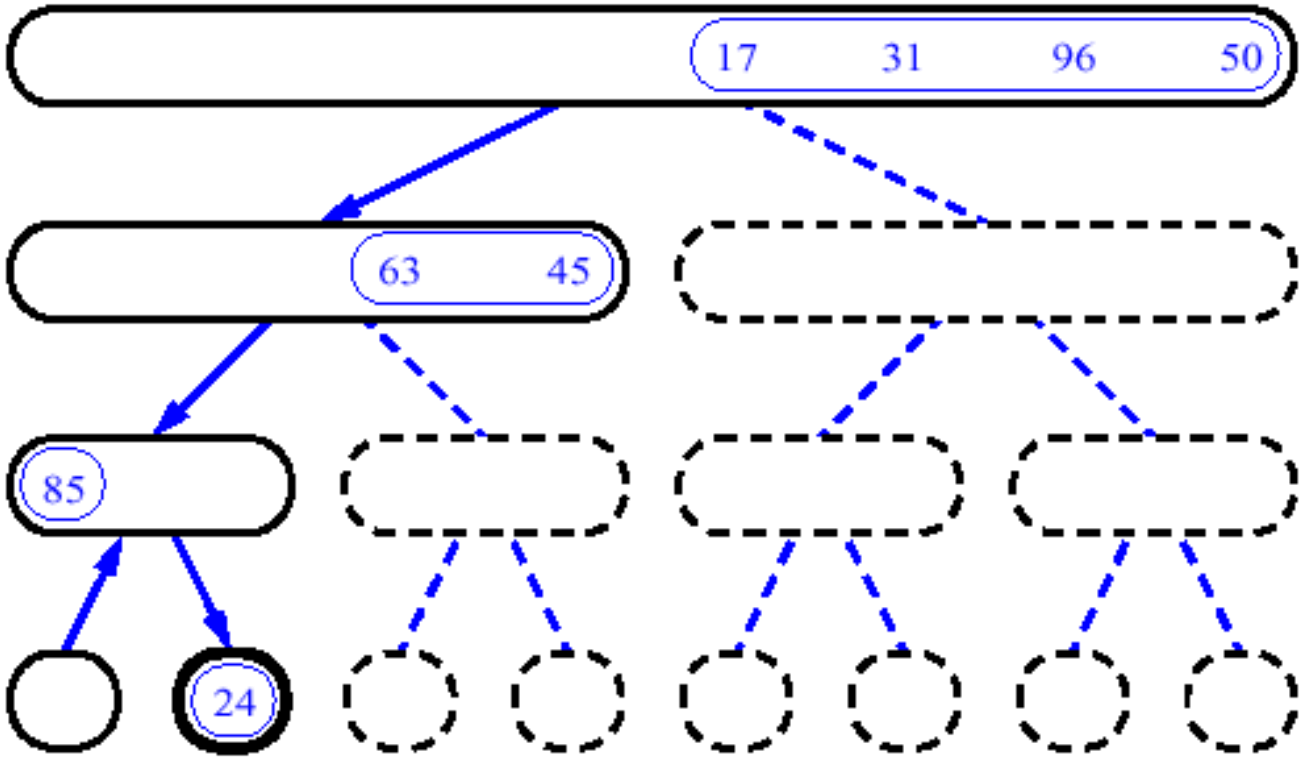
MergeSort (Exemplo) - 4



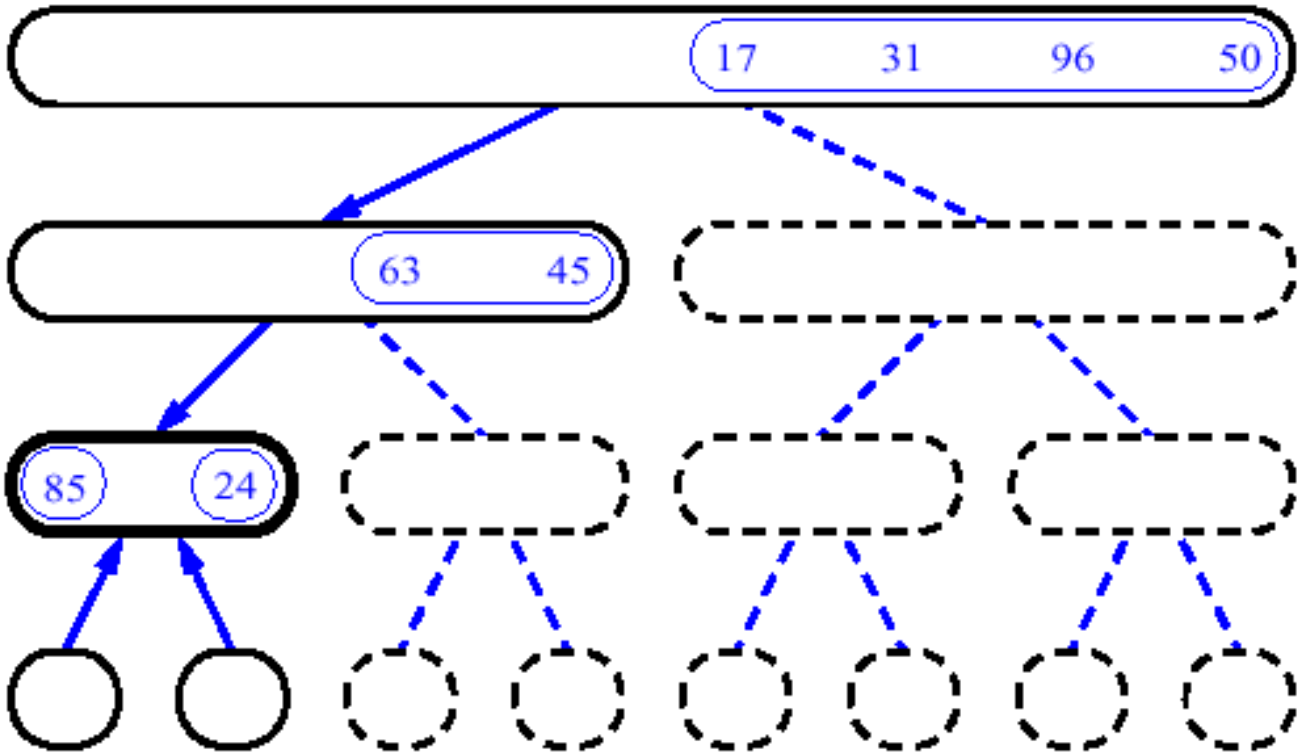
MergeSort (Exemplo) - 5



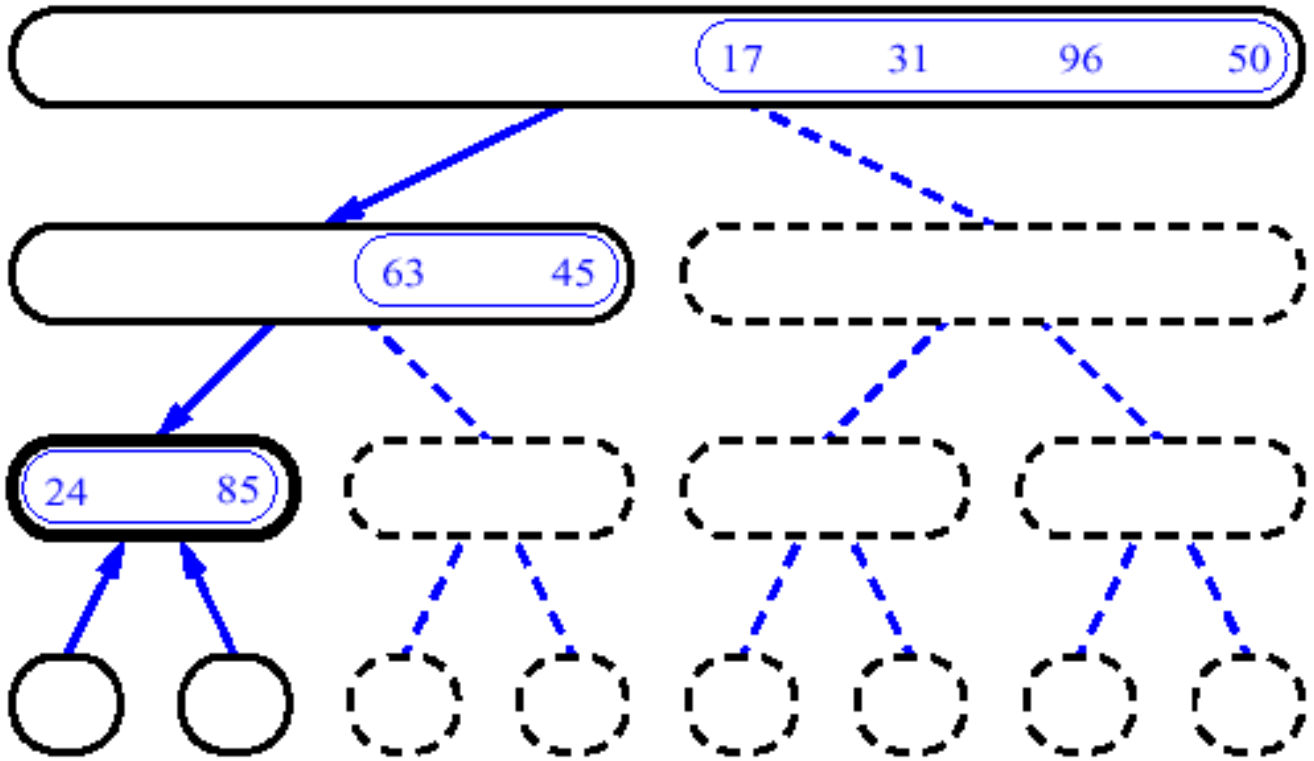
MergeSort (Exemplo) - 6



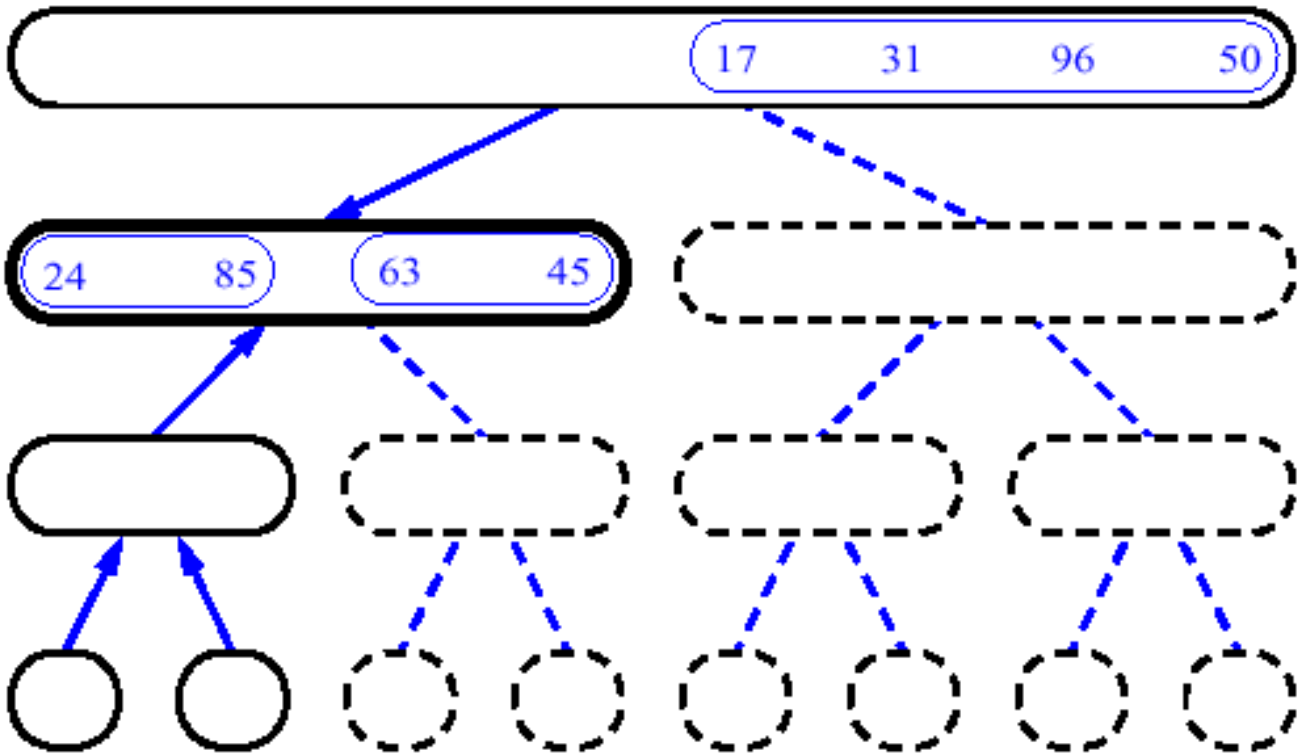
MergeSort (Exemplo) - 7



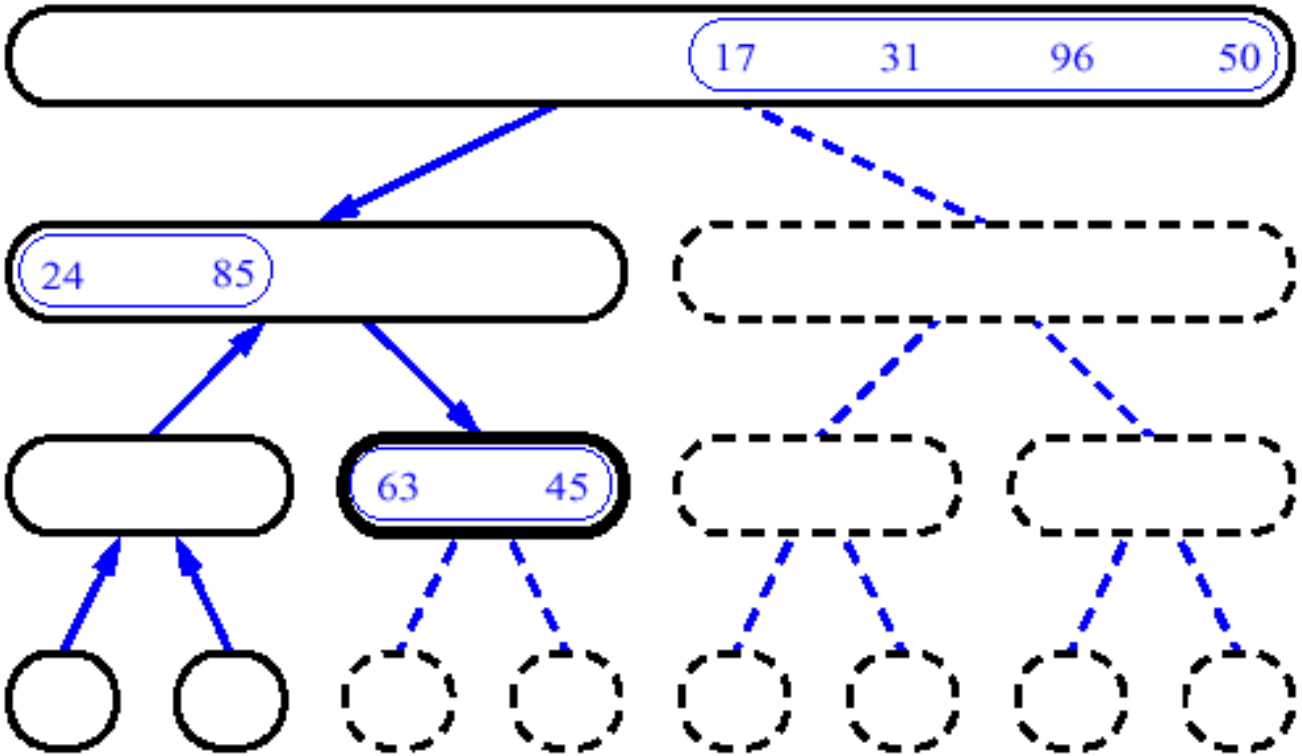
MergeSort (Exemplo) - 8



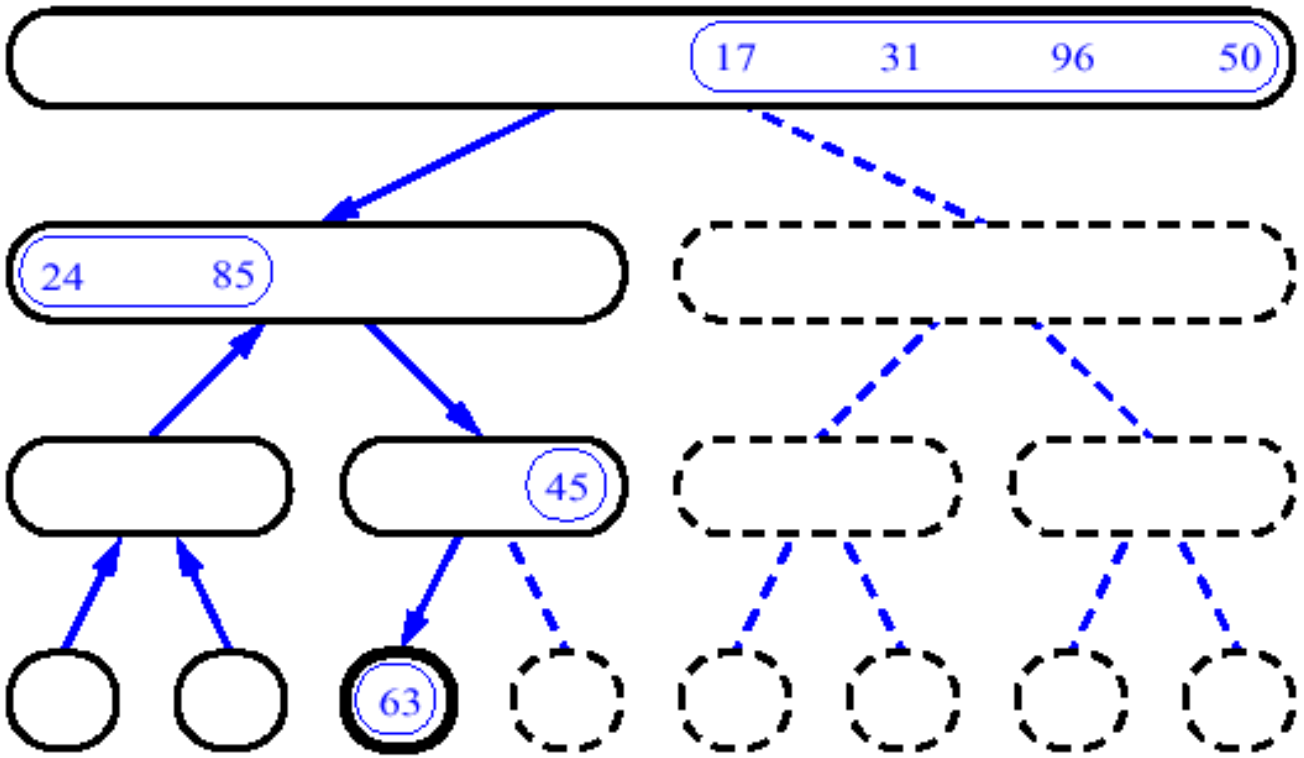
MergeSort (Exemplo) - 9



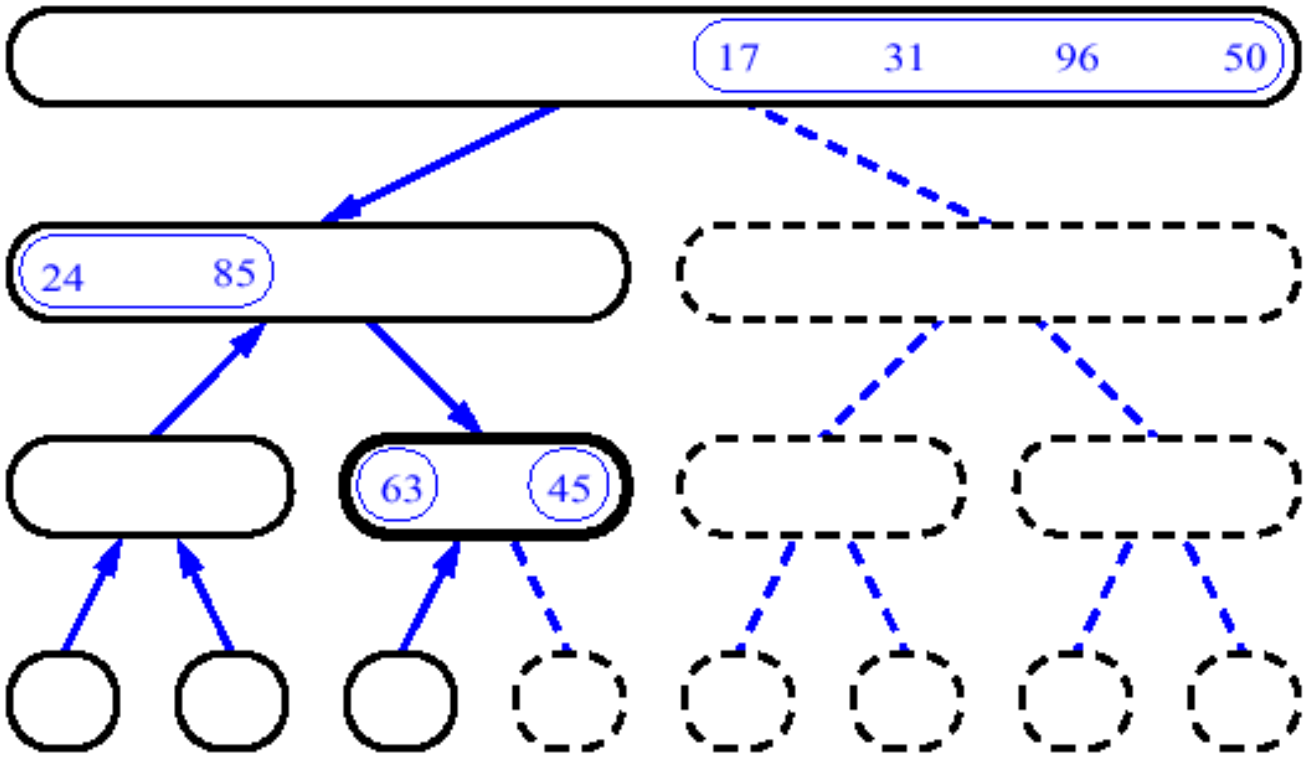
MergeSort (Exemplo) - 10



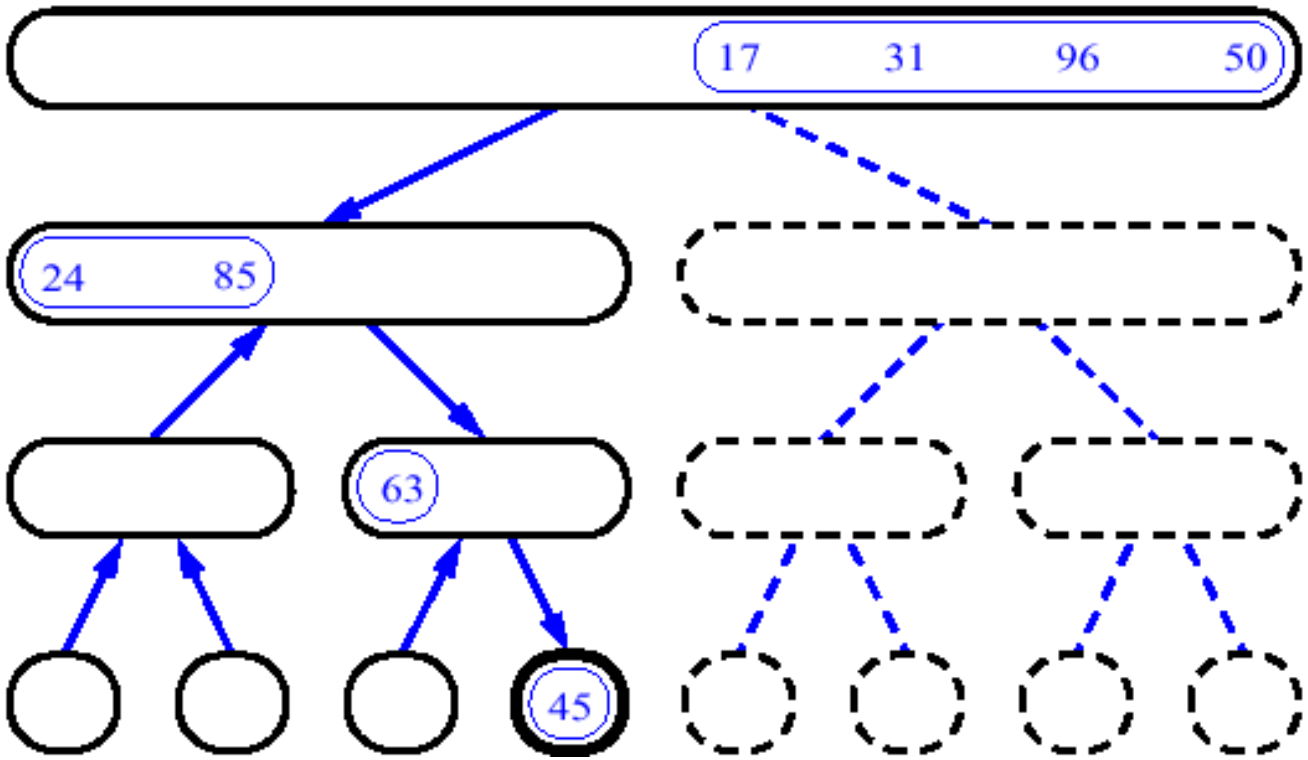
MergeSort (Exemplo) - 11



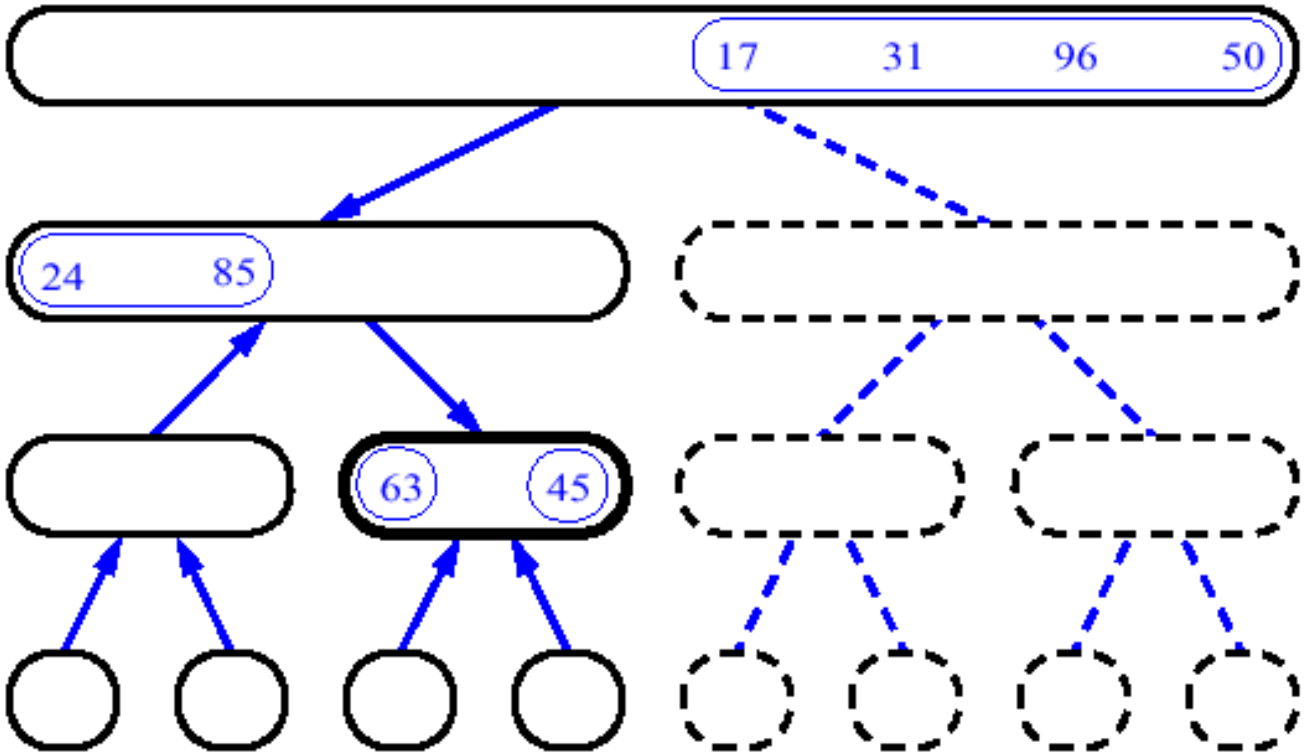
MergeSort (Exemplo) - 12



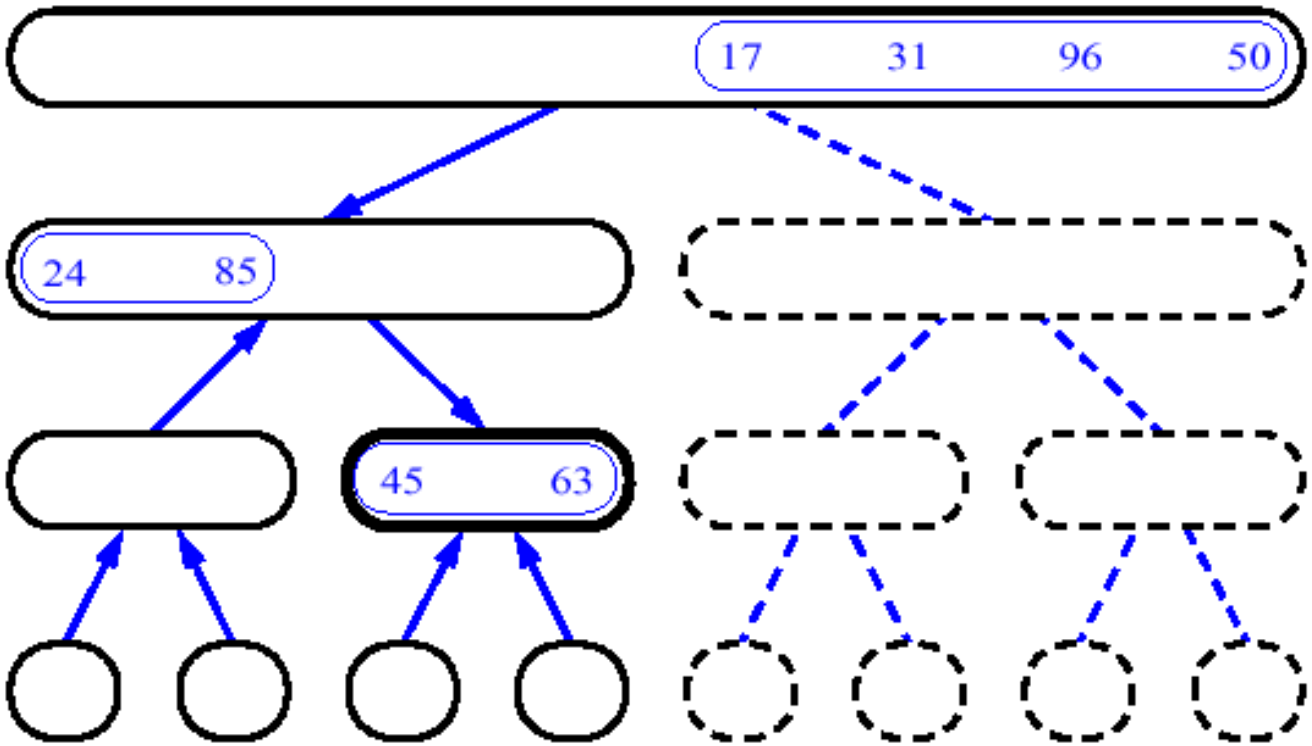
MergeSort (Exemplo) - 13



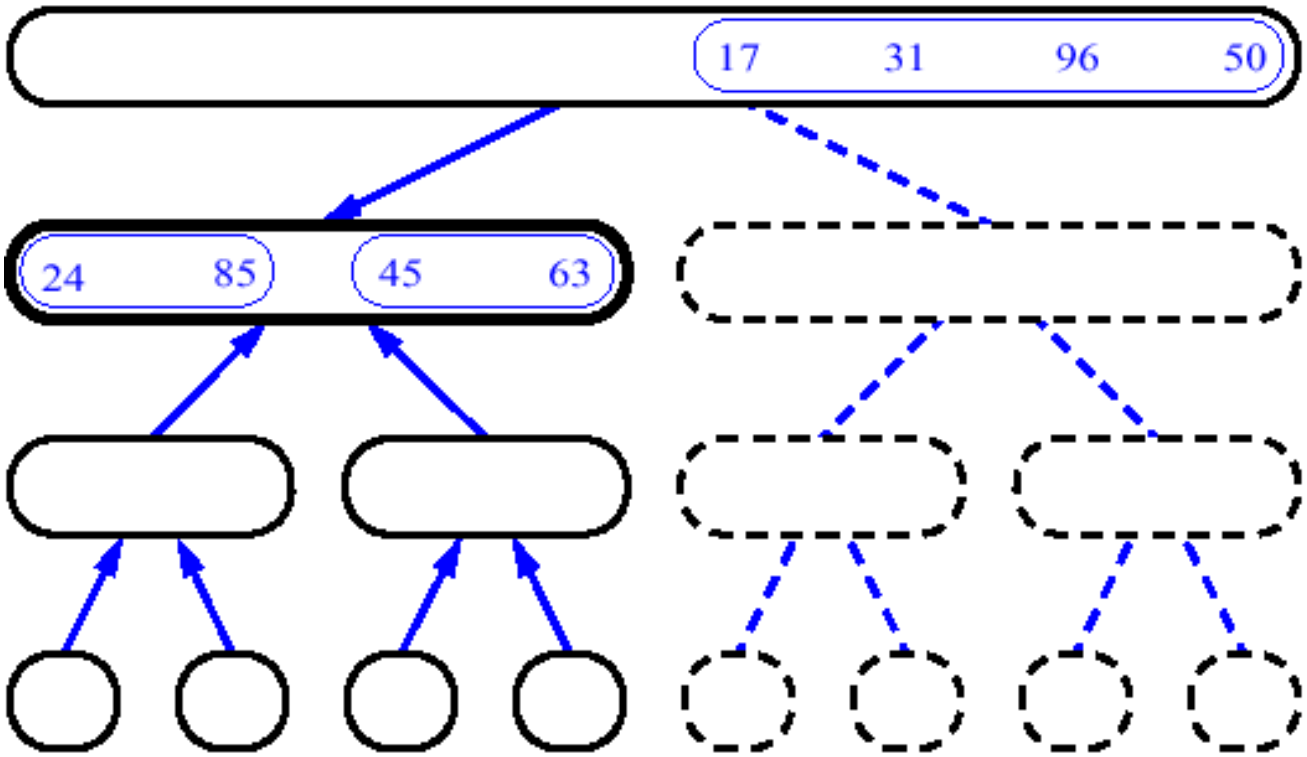
MergeSort (Exemplo) - 14



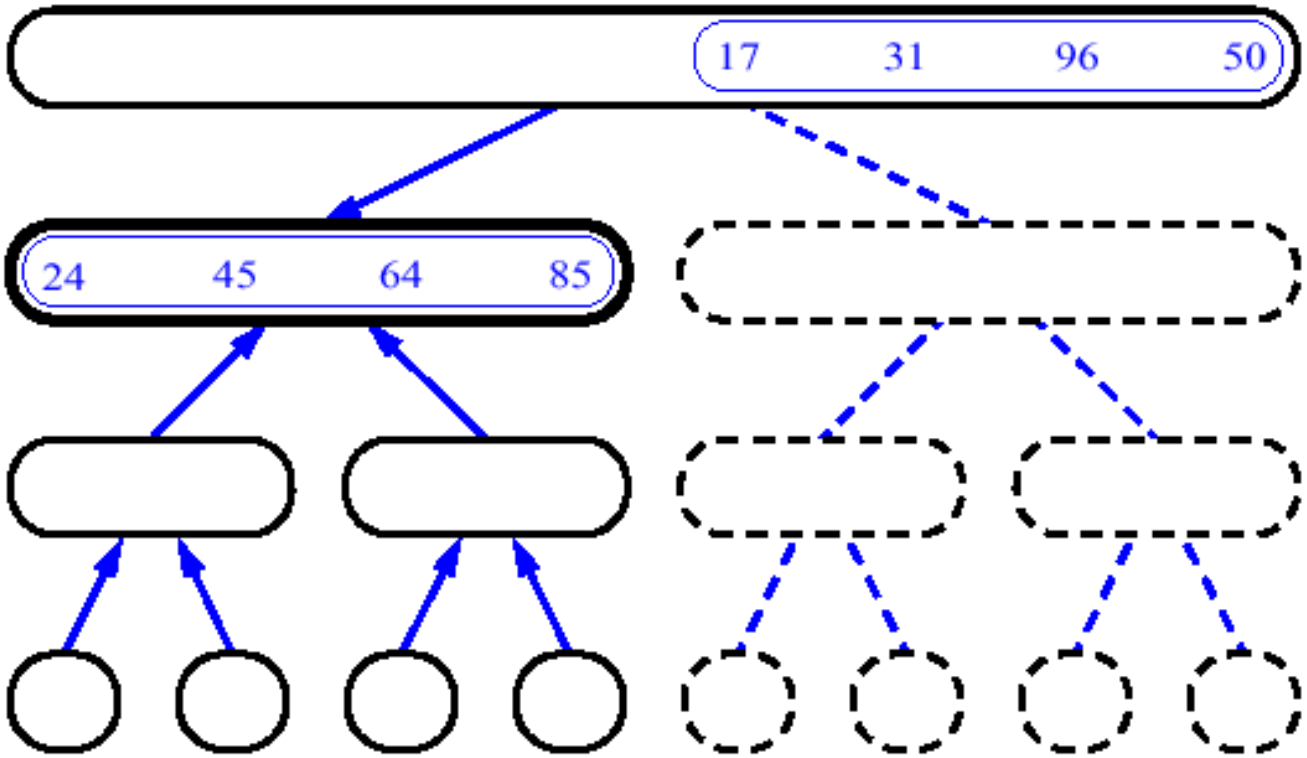
MergeSort (Exemplo) - 15



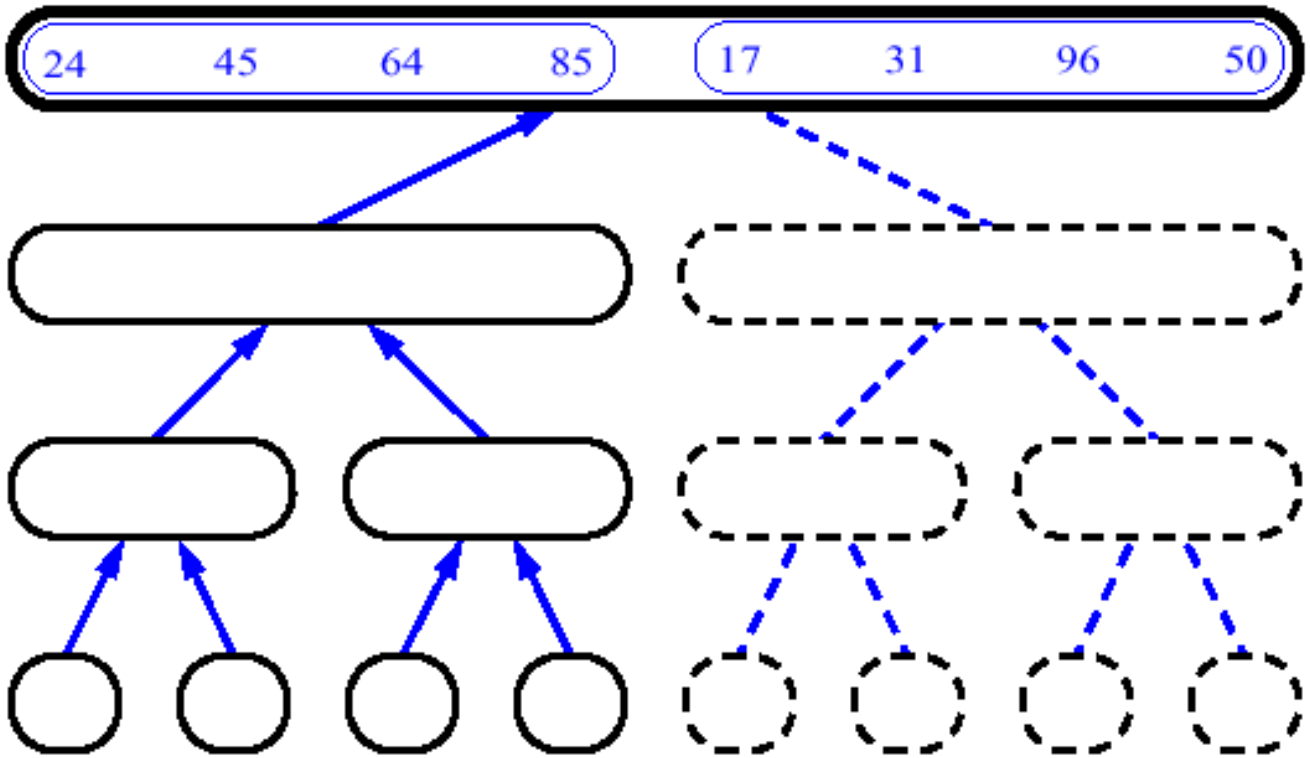
MergeSort (Exemplo) - 16



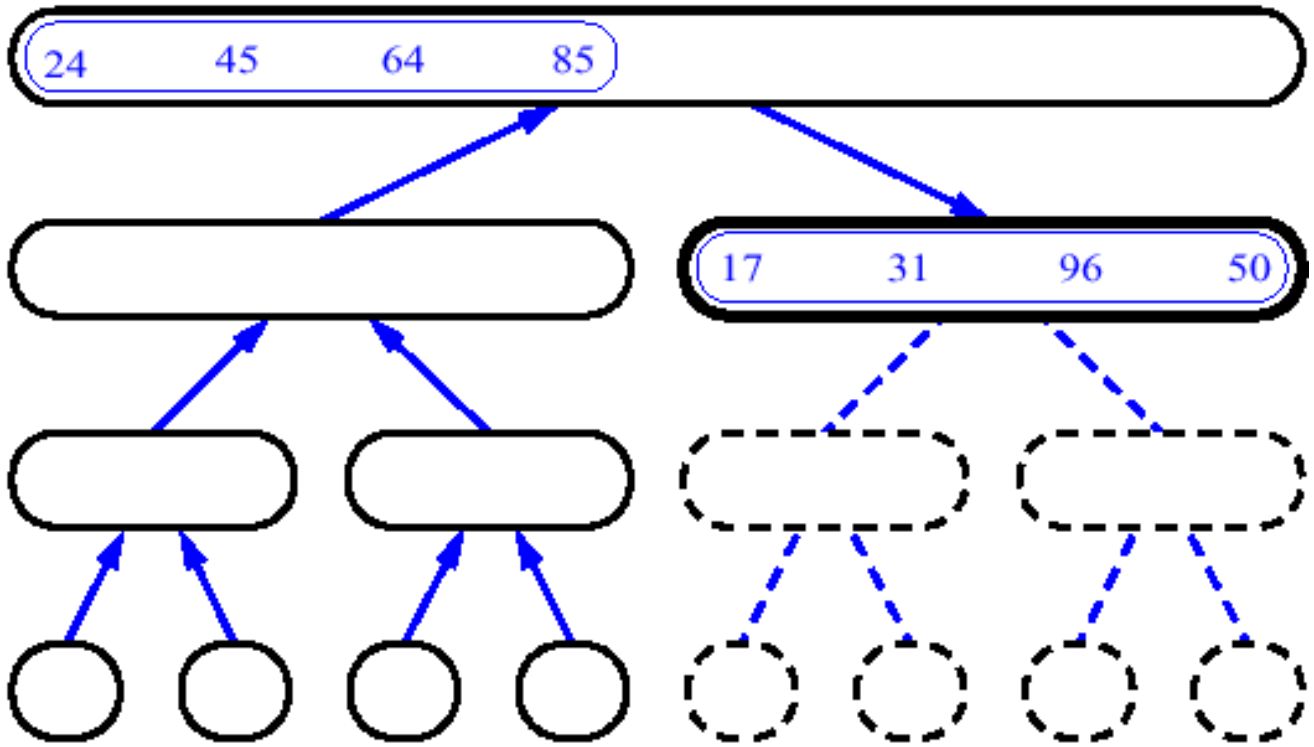
MergeSort (Exemplo) - 17



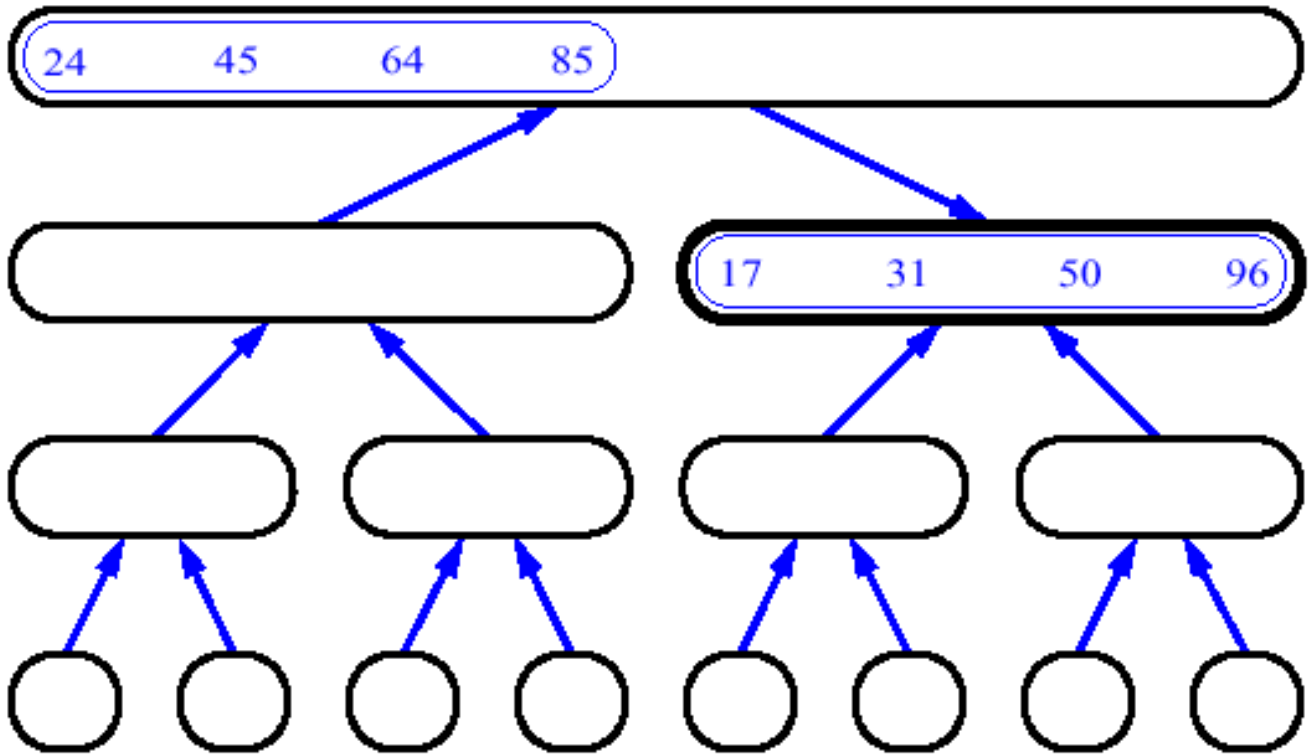
MergeSort (Exemplo) - 18



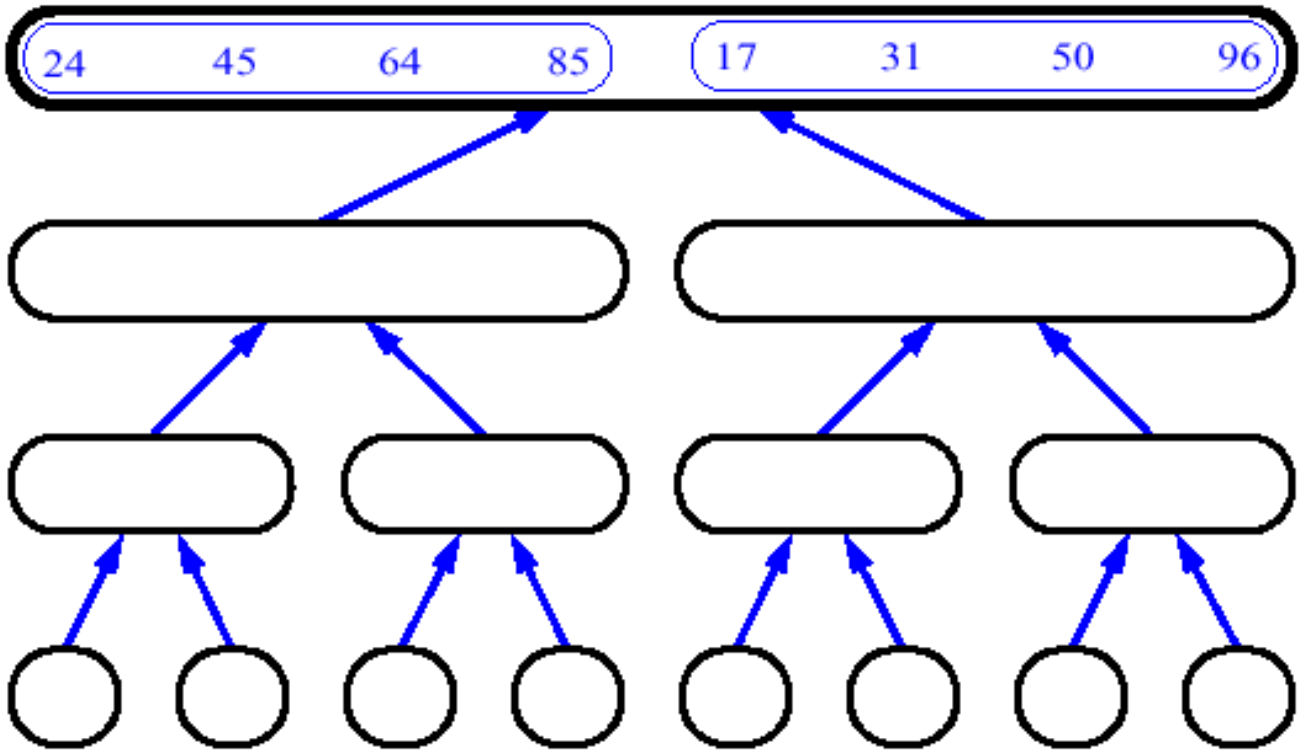
MergeSort (Exemplo) - 19



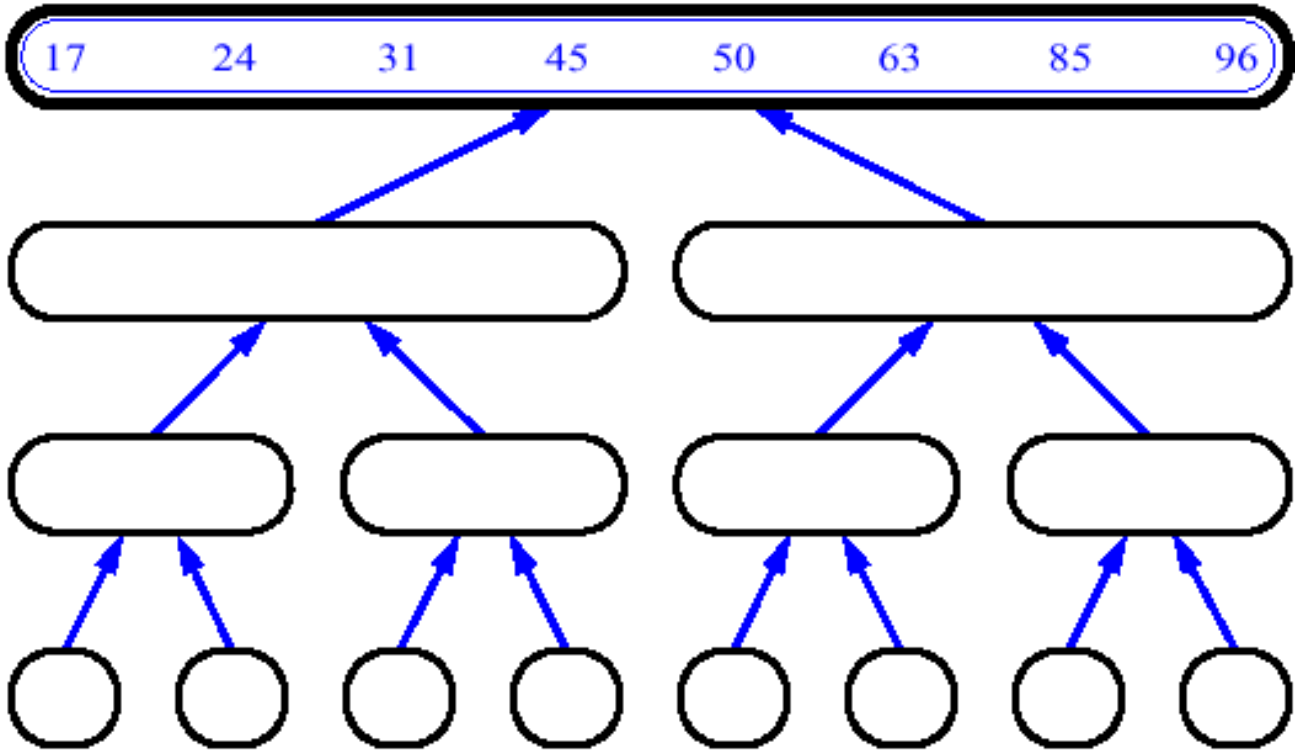
MergeSort (Exemplo) - 20



MergeSort (Exemplo) - 21



MergeSort (Exemplo) - 22



VAMOS APRENDER A RESOLVER AS EQUAÇÕES DE RECORRÊNCIA!

Pergunta

- Fatores constantes somados no custo de um programa recursivo (em cada instância) podem mudar sua complexidade ?

Exemplo: MaxMin

- Problema:
  - Dado um vetor de inteiros A, encontrar o maior e o menor elementos de A
  - O número de comparações entre elementos de A é dado por T(n) = 2 (n-1) para qualquer caso

```
MaxMin(A[1…n])
    Max ← A[1]; Min ← A[1];
    for i = 2 … n
      if A[i] > Max then Max ← A[i];
      if A[i] < Min then Min ← A[i];
    end_for
```

Exemplo: MaxMin (2)

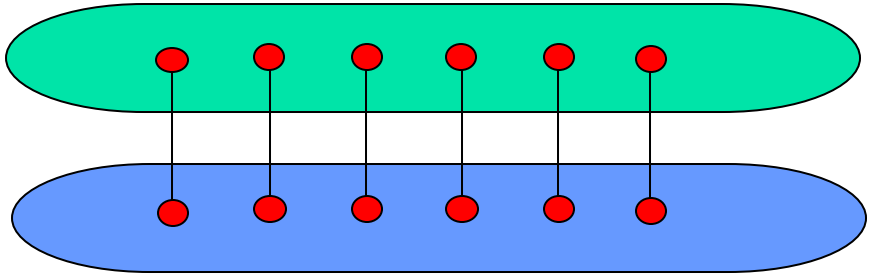
- Versão 2
  - Melhor caso: A está em ordem crescente T(n) = n-1
  - Pior caso: A está em ordem decrescente T(n) = 2(n-1)
  - Caso médio: T(n) = n-1 + (n-1)/2 = 3n/2 – 3/2

```
MaxMin2(A[1…n])
    Max ← A[1]; Min ← A[1];
    for i = 2 … n
      if A[i] > Max then Max ← A[i];
                    else if A[i] < Min then Min ← A[i];
    end_for
```

Exemplo: MaxMin (3)

```
MaxMin3(A[1…n])
   if (n mod 2)>0 then u ← n; A[n+1]=A[n]
                  else u ← n-1;
   if A[1]>A[2] then Max ← A[1]; Min ← A[2]
                else Min ← A[1]; Max ← A[2]
   i ← 3;
   while i ≤ u do
     if A[i]>A[i+1] then
                      if A[i] > Max then Max ← A[i];
                      if A[i+1] < Min then Min ← A[i+1];
                  else
                       if A[i] < Min then Min ← A[i];
                      if A[i+1] > Max then Max ← A[i+1];
     i ← i+2;
  end-while
```

Exemplo MaxMin (4)



$$T(n) = n/2 + (n-2)/2 + (n-2)/2$$
$$= 3n/2 - 2 = O(n)$$